

Day 1 & Day 2: Data Engineering, Pipeline Architecture & Baseline Modeling Report

1. Executive Summary

The primary objective of Days 1 and 2 was to transition an uncurated, synthetically degraded dataset (`adult_income_issues.csv`) into a clean, leakage-resistant data processing and modeling pipeline. The problem revolves around predicting whether an individual earns more than \$50,000 annually (binary classification).

Rather than training models directly on raw extracts, Days 1 and 2 focused on resolving severe data anomalies, implementing robust feature transformations, enforcing entity-level data partitioning, and establishing a baseline gradient-boosted tree model.

AI Disclosure Statement: Large Language Model (LLM) tooling was utilized as an automated pair programmer across the project lifecycle for identifying data corruption patterns, writing scikit-learn transformers, and generating documentation.

2. Day 1: Data Ingestion, Profiling & Anomaly Resolution

Exploratory data analysis of `adult_income_issues.csv` revealed substantial structural issues that prevent out-of-the-box machine learning models from executing:

2.1. Structural Shifts and Value Corruption

- **Column Alignment Errors:** Certain rows contained misaligned columns where string categorical values (such as `"Federal-gov"` or `"Private"`) shifted into continuous numeric columns like `age`. Directly casting these columns to float threw unhandled runtime exceptions (`ValueError: could not convert string to float`).
- **Coerced Parsing Strategy:** Implemented `pd.to_numeric(col, errors='coerce')` to gracefully transform invalid string entries into missing values (`NaN`), preserving valid data points while shielding downstream transformations.

2.2. Target Variable Harmonization

- The ground truth label contained heterogeneous variations and punctuation artifacts (e.g., `>50K`, `>50K.`, `<=50K`, `<=50K.`).

- A deterministic cleaning function stripped whitespace and punctuation, mapping classes to a strict binary representation: $\text{target} \in \{0, 1\}$ where 1 denotes income $> 50\text{K}$ and 0 denotes $\leq 50\text{K}$.

2.3. Entity Identification

- An individual identifier column (`person_id`) was audited. Because multiple records could originate from the same person or entity over time, standard random k-fold or random train/test splits would risk cross-split contamination.

3. Day 2: Modular Pipeline Architecture & Group Partitioning

To guarantee reproducible transformations and prevent data leakage between evaluation sets, all feature engineering was refactored into modular Scikit-Learn components.

3.1. Entity-Aware Data Partitioning

- To prevent duplicate or related individuals from appearing in both training and evaluation subsets, a grouped partition strategy was implemented using `GroupShuffleSplit`:
 - **Train Set:** ~70% (6,487 observations)
 - **Validation Set:** ~15% (1,392 observations)
 - **Test Set:** ~15% (1,392 observations)
- The grouping key was locked to `person_id`.

3.2. ColumnTransformer Preprocessing Design

All transformations were encapsulated in a unified `ColumnTransformer`:

- **Continuous Features (`num_cols`):**
 - `SimpleImputer(strategy='median')`: Replaces missing and coerced values with median statistics computed solely on the training partition.
 - `StandardScaler()`: Normalizes numeric scales for standard distance and gradient behavior.
- **Categorical Features (`cat_cols`):**
 - `SimpleImputer(strategy='constant', fill_value='missing')`: Catches missing or null category codes.
 - `OneHotEncoder(handle_unknown='ignore', sparse_output=False)`: Encodes categorical attributes while gracefully ignoring novel categories during evaluation without raising exceptions.

3.3. Baseline Model Selection

- **Model Type:** LightGBM (`LGBMClassifier`).

- **Rationale:** Gradient boosted trees provide native speed, effective handling of mixed tabular data types, and strong predictive baselines for credit and income scoring tasks.
- **Initial Evaluation Note:** At this stage, candidate features still contained non-causal surrogate variables (`post_adjudication_risk_code` and `db_row_surrogate_key`), which subsequently led to the 1.0 AUC target leakage discovery diagnosed and resolved in Day 4.

4. Pipeline Summary & Operational Checklist

Stage	Component	Technical Implementation	Purpose
Ingestion	Coerced Typecasting	<code>pd.to_numeric(..., errors='coerce')</code>	Eliminate string values in numeric columns
Target Mapping	Binary Binarizer	Custom string normalization <code>\$\to \{0, 1\}\$</code>	Standardize multi-format labels
Splitting	Group Isolation	<code>GroupShuffleSplit(groups=person_id)</code>	Prevent entity contamination across splits
Imputation	Median / Constant	<code>SimpleImputer(strategy='median' / 'constant')</code>	Deterministic handling of nulls
Encoding	One-Hot Encoding	<code>OneHotEncoder(handle_unknown='ignore')</code>	Encode categoricals safely
Classifier	Gradient Boosting	<code>LGBMClassifier(seed=42)</code>	Core tabular prediction baseline